Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No. 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section No.: 12

Team No.: 3

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520080011	K. Samanth	
2520090179	P. Datta Sai Suraj	
2520090079	V. Pavan Kumar	

1. Abstract

Our project, "Linux Inter-Process Communication Using Pipes and FIFOs," focuses on understanding how different processes in a Linux operating system can communicate with each other. Since processes normally have separate memory spaces, they need proper communication mechanisms to exchange data. This project demonstrates two important IPC mechanisms: pipes and FIFOs (named pipes).

In our project, we will use the C programming language and Linux system calls to create processes and establish communication between them. Pipes will be used mainly to demonstrate communication between related processes, such as a parent and child process. FIFOs will be used to show how independent processes can communicate through a named communication channel.

The project will make use of system calls such as fork(), pipe(), mkfifo(), open(), read(), write(), close(), and wait(). Through these implementations, we aim to understand how Linux handles process communication, data transfer, synchronization, and file descriptors.

The final project will provide a simple and practical demonstration of IPC in Linux, helping us understand how processes exchange information and coordinate their activities in an operating-system environment.

2. Problem Statement

In Linux, different processes often need to communicate with each other and exchange information. However, each process normally has its own separate memory space, so one process cannot directly access another process's data.

Our project aims to solve this communication problem by demonstrating how Pipes and FIFOs can be used for communication between processes. We will show how a parent and child process can exchange data using a pipe and how independent processes can communicate using a FIFO.

This project is relevant to Operating Systems and Systems Programming because it helps us understand important concepts such as process creation, inter-process communication, synchronization, file descriptors, and Linux system calls.

3. Objectives

1. To understand and implement Inter-Process Communication (IPC) in Linux using pipes and FIFOs.
2. To demonstrate data communication between parent and child processes using unnamed pipes.
3. To implement communication between independent processes using named pipes (FIFOs).
4. To understand Linux system calls related to process creation, communication, synchronization, and file management.

4. Proposed Methodology

This project is relevant to Operating Systems and Systems Programming because it helps us understand important concepts such as process creation, inter-process communication, synchronization, file descriptors, and Linux system calls.

The project will be developed in a Linux/Ubuntu environment using C programming. First, we will create processes using the fork() system call. For pipe communication, the pipe() system call will be used to create a communication channel between the parent and child processes.

The parent process will write data into the pipe, and the child process will read the data from it. This will help us understand how data can be transferred between related processes.

For FIFO communication, we will create a named pipe using mkfifo(). Two independent processes will then use the FIFO to exchange information. One process will write data while the other process reads it.

8. Expected Outcome

The project is expected to produce a working Linux-based demonstration of Inter-Process Communication using Pipes and FIFOs.

The system will successfully demonstrate:

Creation of parent and child processes.

Communication between related processes using unnamed pipes.

Communication between independent processes using named pipes (FIFOs).

Transfer of messages/data from one process to another.

Use of Linux system calls such as fork(), pipe(), mkfifo(), read(), write(), open(), close(), and wait().

Proper synchronization and resource management between processes.

The final outcome will help users understand how Linux provides IPC mechanisms for exchanging data between processes and how these mechanisms are implemented at the operating-system level.

9. GitHub Repository

GitHub Repository Link: https://github.com/samanthkaturi-spec/KLH-AIDS-CSIT-2026-03-LINUX_IPC_USING_PIPES.git

Repository Created: Yes

Faculty Name and Signature

Faculty Name: M. Raghupathi

Signature:

Remarks:

Faculty Use

Project Approved: Approved

☑ Yes ☐ No

Date: 9/3/2026

Faculty Signature: